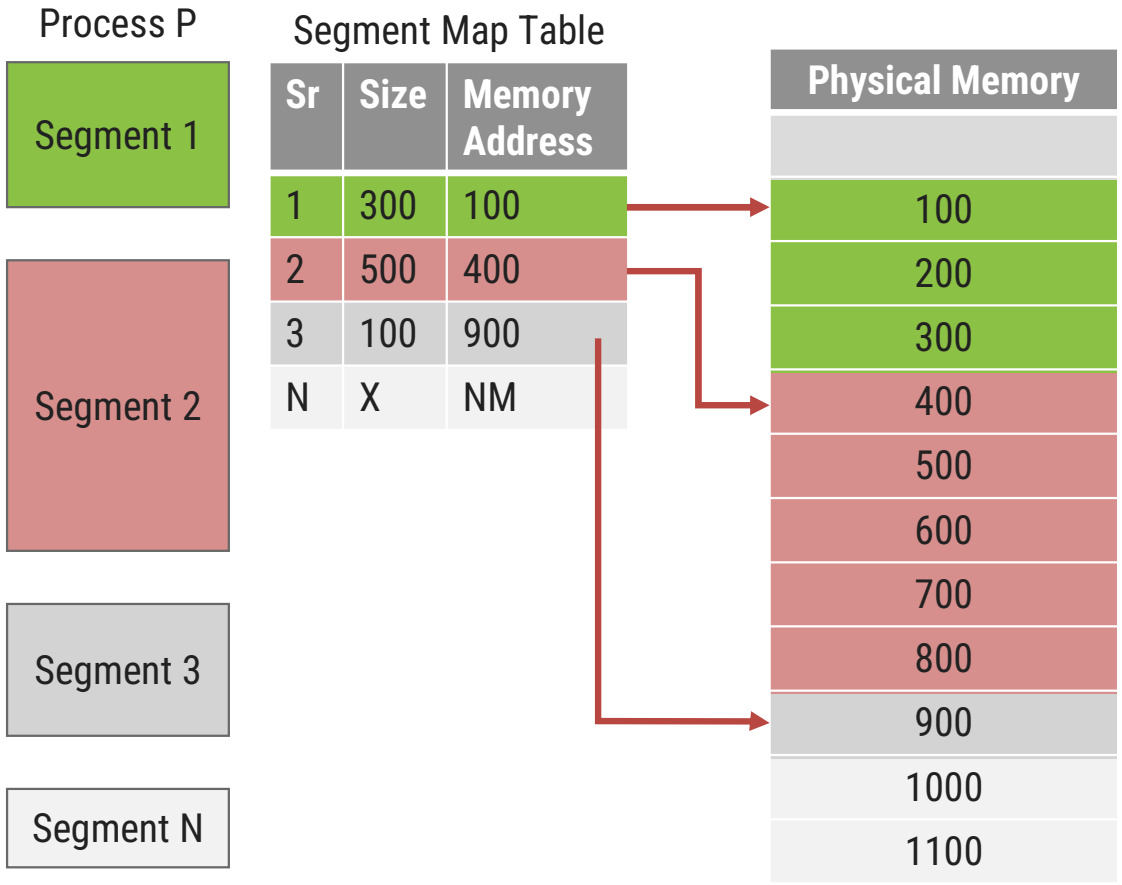
Implementation of Virtual Memory: Segmentation

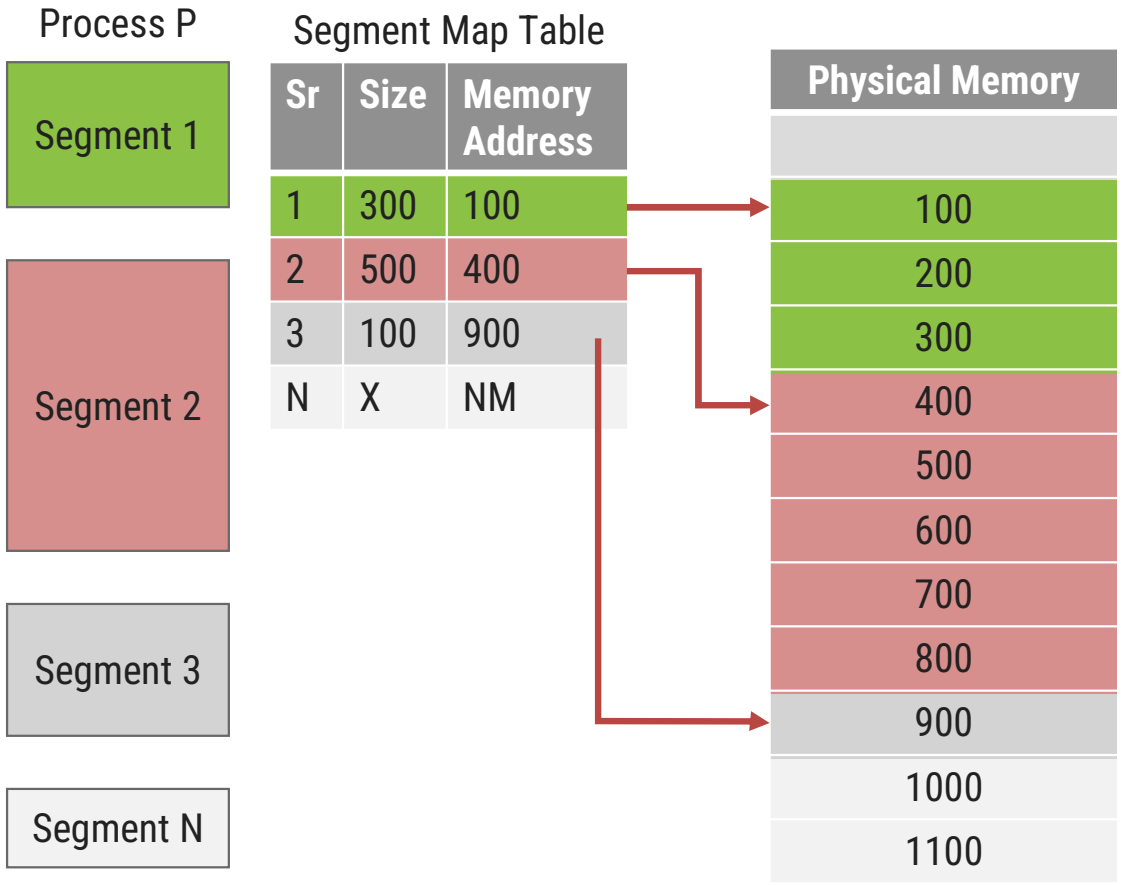
Segmentation

- Segmentation is a **memory management technique in which each job is divided into several segments of different sizes**, one for each module that contains pieces that perform related functions.
- **Each segment is actually a different logical address space** of the program.
- **When a process is to be executed**, its **corresponding segmentation are loaded into non-contiguous memory** though every segment is loaded into a contiguous block of available memory.
- Segmentation memory management works very similar to paging but here **segments are of variable-length where as in paging pages are of fixed size**.



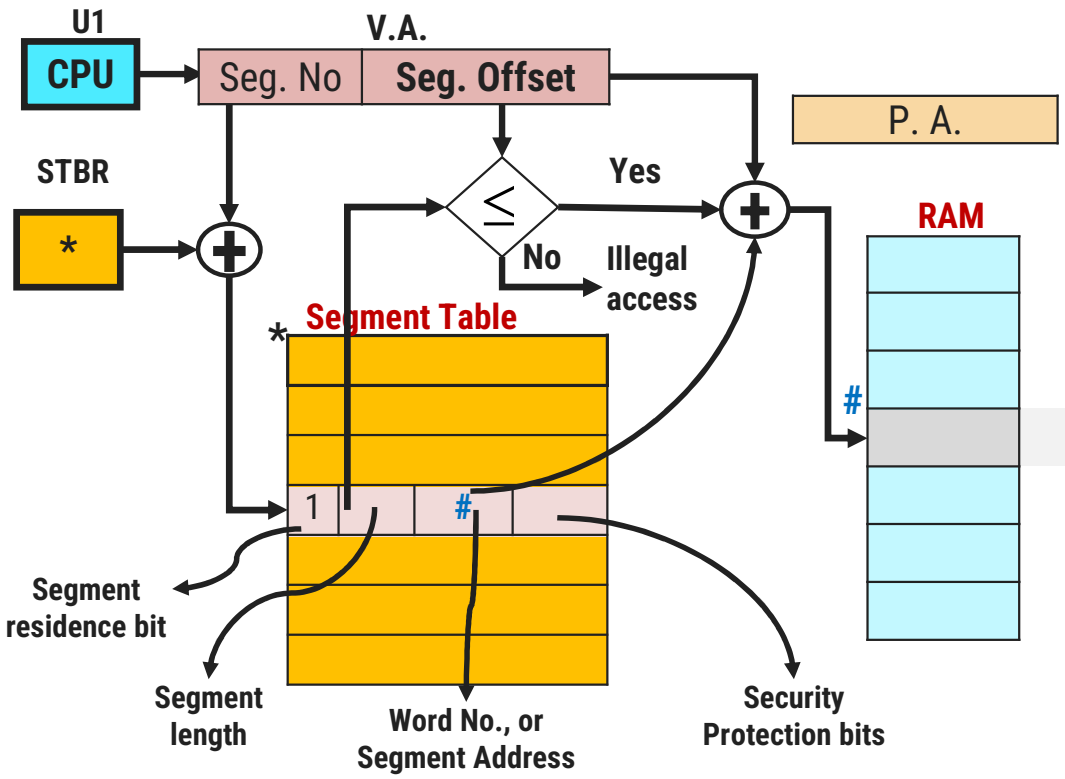
Segmentation

- A program segment contains the program's main function, utility functions, data structures, and so on.
- The **operating system maintains a segment map table for every process**.
- **Segment map table contains list of free memory blocks along with segment numbers, their size and corresponding memory locations** in main memory.
- For each segment, the **table stores the starting address of the segment and the length of the segment**.
- A reference to a memory location includes a value that identifies a segment and an offset.



Virtual Memory Implementation: Segmentation

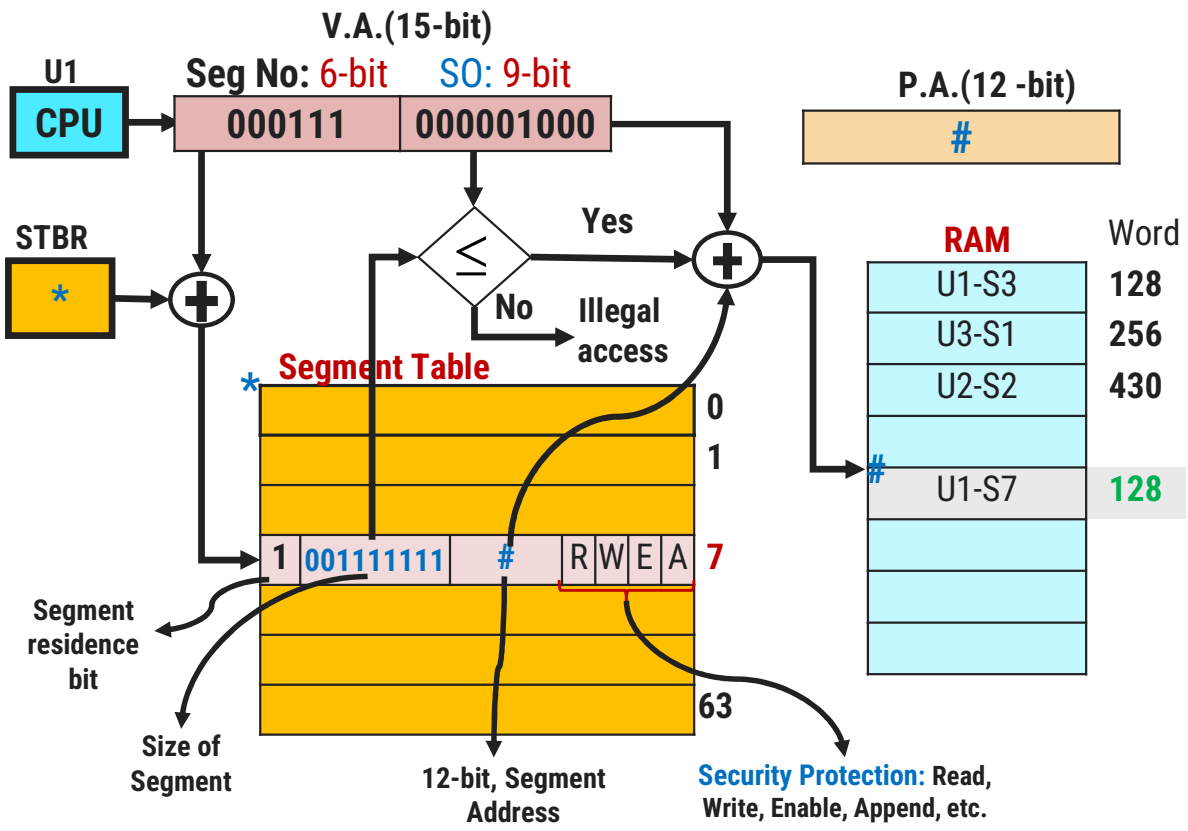
- VM is divided into variable-sized segments, and a segment is a collection of words.
- RAM is also a collection of words. One segment is occupied in contiguous memory in RAM.
- Two segments can be in non-contiguous.
- **Address translation:**
  - Find the segment table in RAM and read the entry.
  - Check the segment size and find the segment address in RAM.
  - Use the address and segment offset to map this LA to PA.
  - **STBR:** **Se**gment table base register. Store base or starting address of the segment table. (i.e., Word address).



- Though segments are variable-sized, the maximum size of a segment is there.
- Segment table maps VA to PA.

Virtual Memory Implementation: Segmentation Example

- VM is divided into variable-sized segments, and a segment is a collection of words.
- RAM is also a collection of words. One segment is occupied in contiguous memory in RAM.
- **Address translation:**
  - Find the segment table in RAM and read the entry.
  - Check the segment size and find the segment address in RAM (**12-bit**).
  - Use the address and segment offset to map this LA to PA.
  - **If the referred segment is not found, then a segment fault occurs**.
- *Here, no internal fragmentation.*
- External fragmentation may be more than the internal fragmentation occurred in paging.
- To avoid external fragmentation, we use paged segmentation.



Virtual Memory Implementation: Segmentation Exercise

- Ques. Consider VM implementation using segmentation, where VA is 24 bits, and the number of segments is 1024. RAM is of 20-bits address. Calculate:
  - Maximum size of any segment.
  - No of entries in the segment table.
  - Size of segment table.

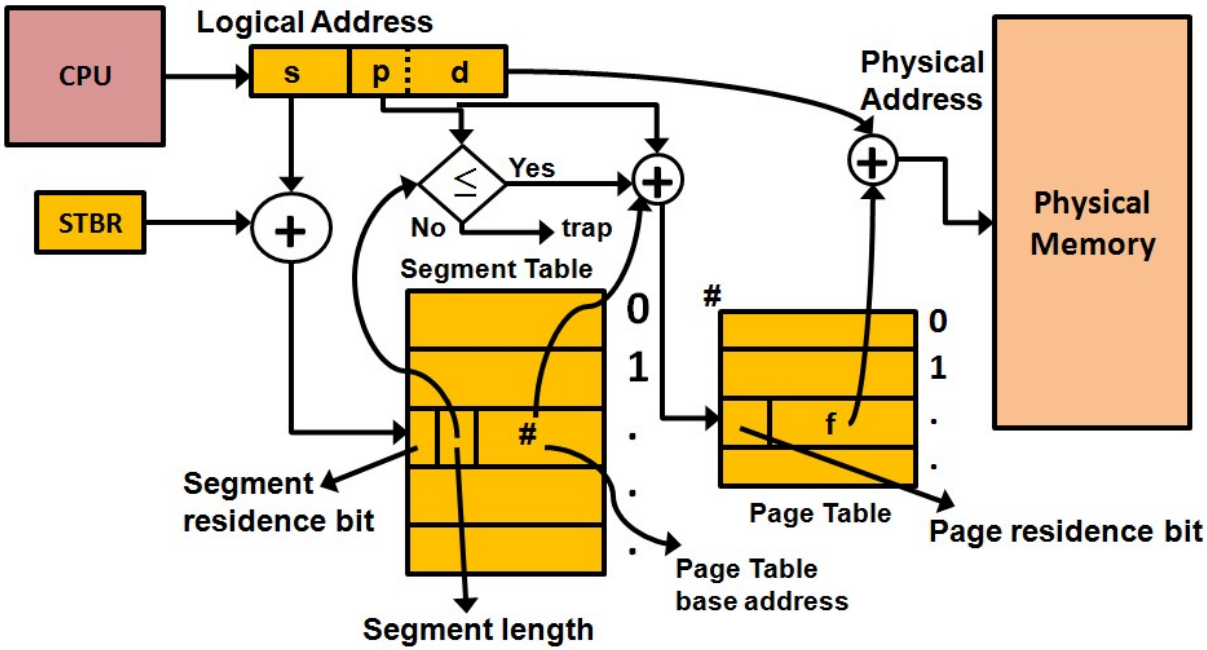
- Ans.
  - Maximum size of any segment: 16384.
  - No of entries in the segment table: 1024.
  - Size of segment table: 5 KB.
    - 1 entry = 1 + 20 + 14 + 4
    - 1 entry Size = 39 bits = 5 Bytes.
    - Size of Table = 1024 x 5 Bytes
    - Size of table = 5 KB.

Virtual Memory Implementation: Paged Segmentation

- Here, the segment is divided into fixed-size blocks, i.e., pages, and a page is a collection of words.
- RAM is divided into equal-sized frames, which are collections of words. The page and the frame are of same-size.
- Though a segment can be in a non-contiguous space, a page must be in a contiguous space.
- Note: we can carry a half or part of the segment in the memory. (Part: inform of pages).

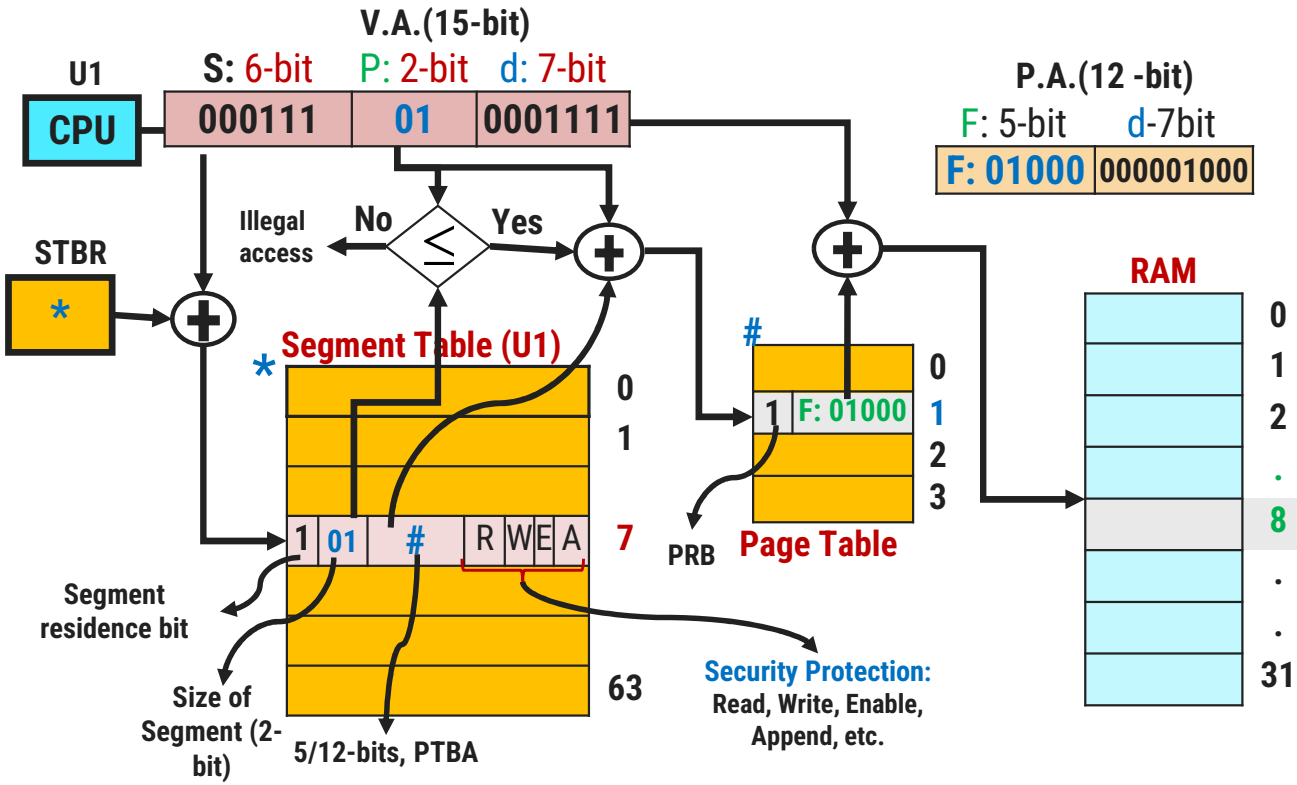
- **Address translation:**
  - Find the segment table in RAM and read the entry.
  - Check the segment size and find the page table address in RAM.
  - Read the page table entry, find **f**, and use **f** and **d** to map this LA to PA.

- Segment and Page table map VA to PA.



Virtual Memory Implementation: Paged Segmentation

- Here, the segment is divided into fixed-size blocks, i.e., pages, and a page is a collection of words.
- **Address translation:**
  - Find the segment table using STBR in RAM and read the segment (**S**) entry.
  - Check the segment size and find the page table base address (PTBA) in RAM.
  - Read the page table entry, find **F**, and use **F** and **d** to map this LA to PA.
  - *If the segment or page is not found, then a segment or page fault occurs.*
- No external fragmentation. **Internal fragmentation may be, but less than segmentation and more than paging.**
- **Three memory cycles required.**



Note: Maximum Internal fragmentation = degree of multiprogramming x No. of users.

Exercises: Segmentation

- *Consider 28-bit VA & 2048 segments are implemented using paged segmentation. The physical address is 20 bits, and the frame size is 256 words. Calculate*
  - *Number of frames in RAM*
  - *Maximum size of any segment.*
  - *Number of entries & size (in KB) of segment table*
  - *Number of entries and size (in KB) of the page table of any segment.*

- ***Ans.***
  - *Number of frames in RAM:* ***4096***
  - *Maximum size of any segment:* ***512 pages***
  - *Number of entries & size (in KB) of segment table:* ***2048 and 8 KB***
  - *Number of entries and size (in KB) of the page table of any segment:* ***512 and 1 KB.***

Paging VS Segmentation

Paging	Segmentation
Paging was invented to get large address space without having to buy more physical memory.	Segmentation was invented to allow programs and data to be broken up into logically independent address space and to add sharing and protection.
The programmer does not aware that paging is used.	The programmer is aware that segmentation is used.
Procedure and data cannot be distinguished and protected separately.	Procedure and data can be distinguished and protected separately.
Change in data or procedure requires compiling entire program.	Change in data or procedure requires compiling only affected segment not entire program.
Sharing of different procedures not available.	Sharing of different procedures available.

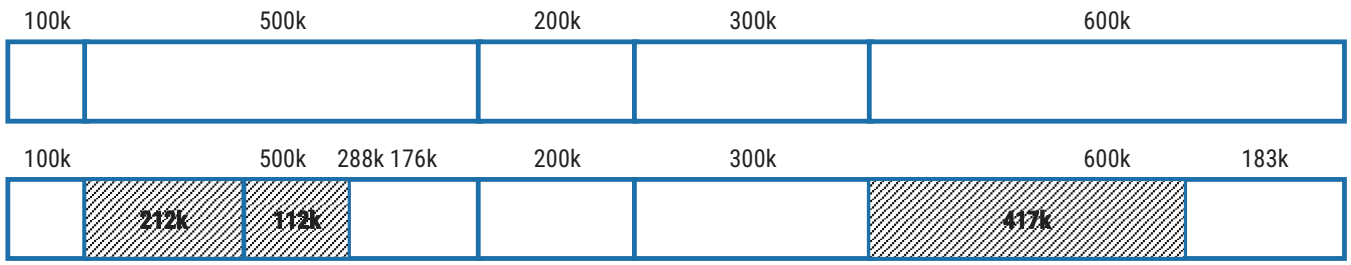
Memory allocation

Memory allocation

- **Four memory allocation algorithms are as follows:**
  - Placement Techniques: Allocate the new process/page to the empty location based on the following strategies. Used with variable- or fixed-sized partitions.
    1. First fit
    2. Next fit
    3. Best fit
    4. Worst fit
  - Replacement Techniques: Allocate the new process/page by replacing the old one using following methods.
    1. Local Page replacement
       - First in-First Out(FIFO)
       - Least Recently used (LRU)
       - Least frequently used (LFU)
       - Optimal
       - 2nd Chance
       - Enhance 2nd Chance
    2. Global page replacement
       - Working Set model

Placement Techniques: First fit

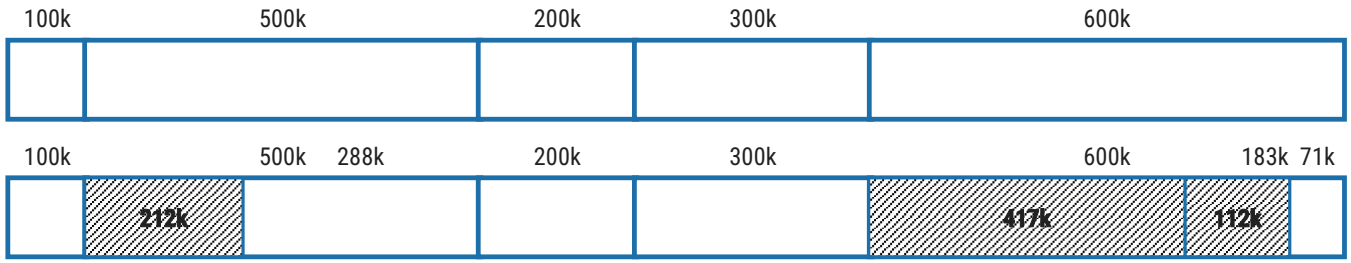
- Search **starts from the starting location** of the memory.
- **First available hole that is large enough to hold the process is selected** for allocation.
- The **hole is then broken up into two pieces**, one **for process** and another **for unused memory**.
- Example: Processes of **212K, 417K, 112K** and **426K** arrives in order.



- Here process of size **426k will not get any partition** for allocation.
- **Fastest algorithm** because it searches as little as possible.
- **Memory loss is higher**, as very large hole may be selected for small process.

Placement Techniques: Next fit

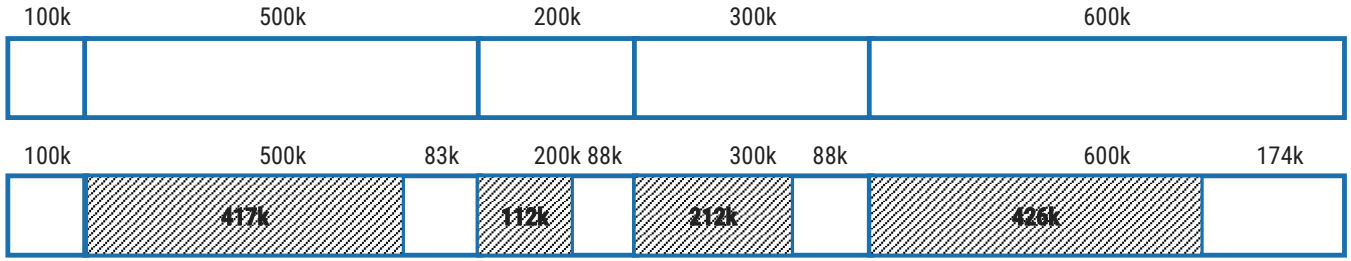
- It works in the same way as first fit, except that **it keeps the track of where it is whenever it finds a suitable hole**.
- The **next time when it is called to find a hole, it starts searching the list from the place where it left off last time**.
- Example: Processes of **212K, 417K, 112K** and **426K** arrives in order.



- Here process of size **426k will not get any partition** for allocation.
- **Search time is smaller.**
- Memory manager must have to **keep track of last allotted hole to process**.
- It **gives slightly worse performance** than first fit.

Placement Techniques: Best fit

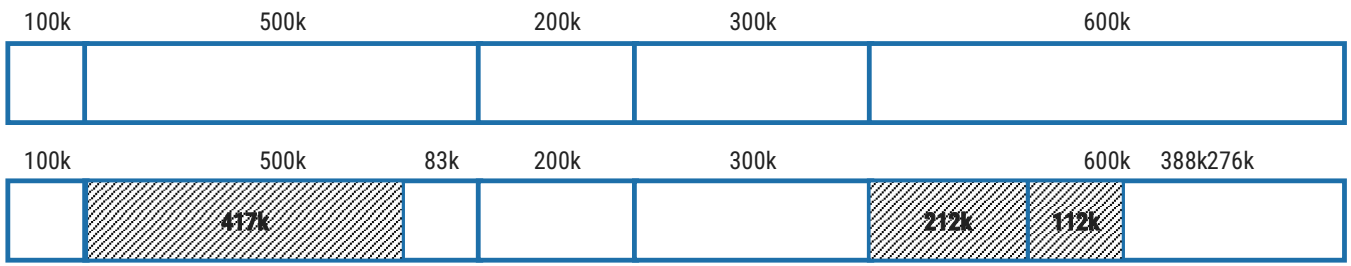
- **Entire memory is searched** here.
- The **smallest hole, which is large enough to hold the process, is selected** for allocation.
- Example: Processes of **212K, 417K, 112K** and **426K** arrives in order.



- **Search time is high**, as it searches entire memory every time.
- **Memory loss is less**.

Placement Techniques: Worst fit

- **Entire memory is searched** here also. The **largest hole, which is largest enough to hold the process, is selected** for allocation.
- Example: Processes of **212K, 417K, 112K** and **426K** arrives in order.



- Here process of size **426k will not get any partition** for allocation.
- **Search time is high**, as it searches entire memory every time.
- This algorithm can be **used only with dynamic partitioning.**

Note: In all the techniques, none is best. It depends on the conditions. We cannot say which method is best.

Page Replacement Algorithms

Page replacement algorithms

- The following are different types of page replacement algorithms

1. Local Page replacement: **The replaced page belongs to the same process that is demanding the new page.**
   - **First in-First Out(FIFO):** Replaces the page that came first.
   - **Least Recently used (LRU):** Replaces the page that is least recently used (not referred to for a long time).
   - **Least frequently used (LFU):** Replaces the page that is least frequently used (referred to very few times)
   - **Optimal:** Looks in the future, replaces the page that will not be referred to for a long time.
   - **2nd Chance:** Replaces the pages based on the reference bit.
   - **Enhance 2nd Chance or NRU:** Replaces the pages based on reference and modified bits.

2. Global page replacement: **Page to be replaced may be of any process.**
   - Working Set model

FIFO Page Replacement Algorithm

- The first in first out page replacement algorithm is the simplest page replacement algorithm.
- The operating system **maintains a list of all pages currently in memory, with the most recently arrived page at the tail and least recent at the head**.

- On a page fault, the **page at head is removed and the new page is added to the tail**.
- When a page replacement is **required the oldest page in memory needs to be replaced**.

- The performance of the FIFO algorithm is **not always good because it may happen that the page which is the oldest is frequently referred by OS**.
- Hence removing the **oldest page may create page fault again**.

FIFO Page Replacement Algorithm

- Page Reference String:
  - 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
  - Three frames

Page Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame – 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
Frame – 2		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
Frame – 3			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
Page Faults (15)	F	F	F	F		F	F	F	F	F	F			F	F			F	F	F

LRU (Least Recently Used) Page Replacement Algorithm

- A good approximation to the optimal algorithm is based on the observation that pages that have been heavily used in last few instructions will probably be heavily used again in next few instructions.
- When page fault occurs, **throw out the page that has been used for the longest time**. This strategy is called LRU (Least Recently Used) paging.
- To fully implement LRU, it is necessary to maintain a linked list of all pages in memory, with the **most recently used page at the front and the least recently used page at the rear**.
- The list must be updated on every memory reference.
- Finding a page in the list, deleting it, and then moving it to the front is a very time consuming operations.

LRU (Least Recently Used) Page Replacement Algorithm

- Page Reference String:
  - 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
  - Three frames

Page Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame – 1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
Frame – 2		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
Frame – 3			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
Page Faults (12)	F	F	F	F		F		F	F	F	F			F		F		F		

Optimal Page Replacement Algorithm

- The moment a page fault occurs, some set of pages will be in the memory.
- One of these pages will be referenced on the very next instruction.
- Other pages may not be referenced until 10, 100, or perhaps 1000 instructions later.

- Each **page can be labeled with the number of instructions that will be executed before that page is first referenced**.
- The optimal page algorithm simply says that the **page with the highest label should be removed**.

- The only problem with this algorithm is that it is **unrealizable**.
- At the time of the page fault, the **operating system has no way of knowing when each of the pages will be referenced next**.

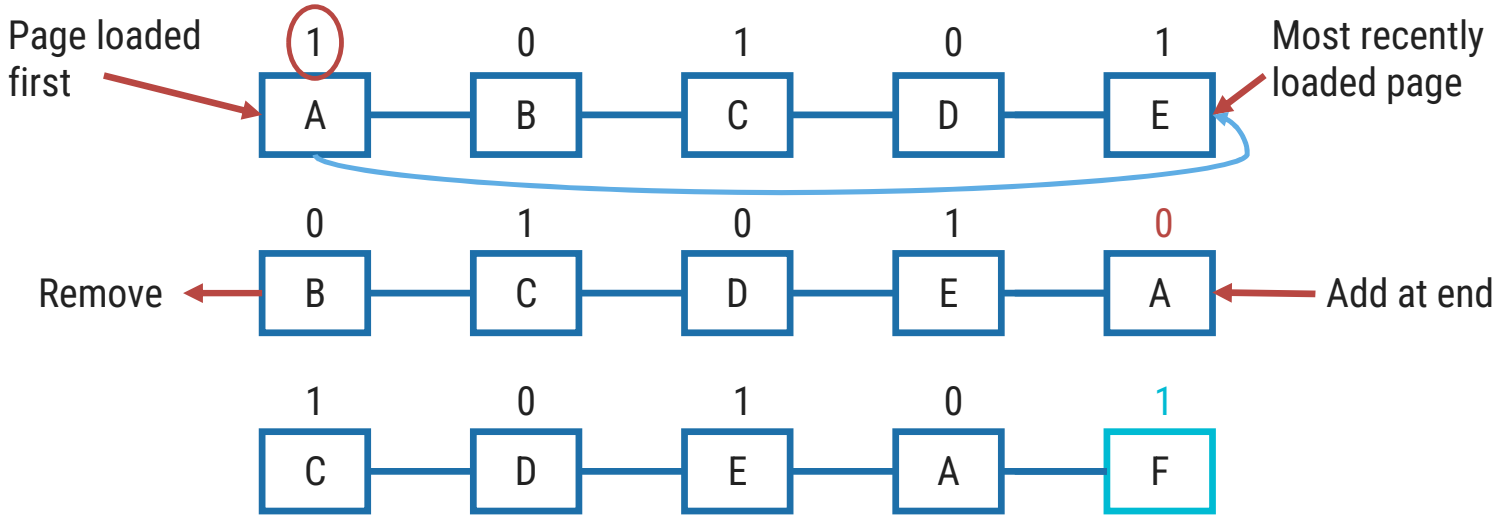
Optimal Page Replacement Algorithm

- Page Reference String:
  - 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 0, 2, 0, 1, 7, 0, 1
  - Three frames

Page Request	7	0	1	2	0	3	0	4	2	3	0	3	2	0	2	0	1	7	0	1
Frame – 1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1
Frame – 2		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
Frame – 3			1	1	1	3	3	3	3	3	3	3	3	3	3	3	3	7	7	7
Page Faults (9)	F	F	F	F		F		F			F						F	F		

Second Chance Page Replacement Algorithm

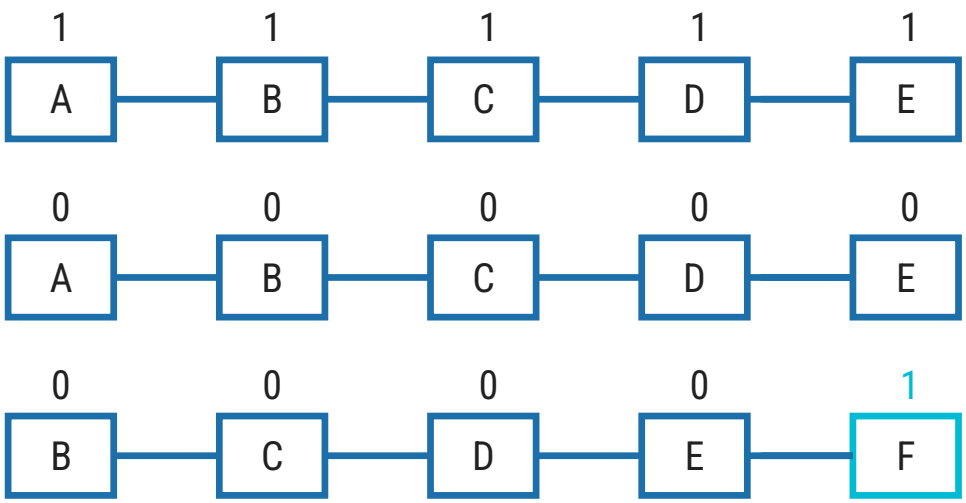
- It is **modified form of the FIFO page replacement algorithm**.
- It looks at the front of the queue as FIFO does, but instead of immediately paging out that page, it **checks to see if its referenced bit is set**.
  - **If it is not set (zero), the page is swapped out**.
  - **Otherwise, the referenced bit is cleared**, the **page is inserted at the back of the queue** (as if it were a new page) and this process is repeated.



Second Chance Page Replacement Algorithm

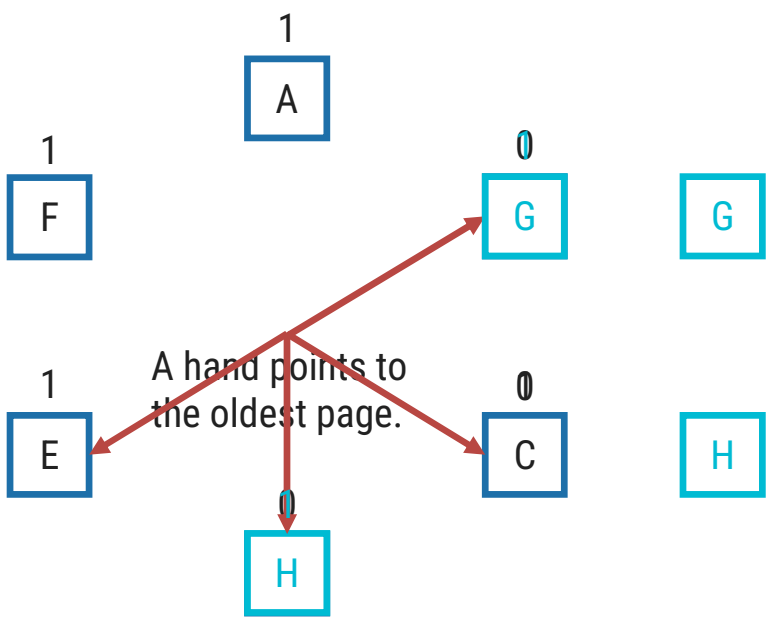
- If all the pages have their referenced bit set, on the second encounter of the first page in the list, that page will be swapped out, as it now has its referenced bit cleared.

- If all the pages have their reference bit cleared, then second chance algorithm degenerates into pure FIFO.



Clock Page Replacement Algorithm

- In second chance algorithm **pages are constantly moving around on its list**. So it is **not working efficiently**.
- A better approach is to **keep all the page frames on a circular list in the form of a clock**.
- When a page fault occurs, the **page being pointed to by the hand is inspected**.
  - If its **R bit is 0**, the page is evicted, the **new page is inserted into the clock in its place**, and the **hand is advanced one position**.
  - If **R is 1**, it is **cleared and the hand is advanced to the next page**.



Enhanced 2nd Chance or NRU (Not Recently Used)

- NRU makes approximation to replace the page based **on R (referenced) and M (modified or dirty) bits**.
- When a process is started up, **both page bits for all pages are set to 0 by operating system**.
- Periodically, the **R bit is cleared**, to distinguish pages that have not been referenced recently from those that have been.

- When page fault occurs, the operating system inspects all the pages and **divide them into 4 categories based on current values of their R and M bits**
  - Case 0 : not referenced, not modified
  - Case 1 : not referenced, modified
  - Case 2 : referenced, not modified
  - Case 3 : referenced, modified

Most Suitable for Replacement			
Page	R-bit	M-bit	Order
Page-0	0	0	↑
Page-1	0	1	
Page-2	1	0	
Page-3	1	1	

- The NRU (Not Recently Used) algorithm removes a page at random from the lowest numbered nonempty class.

NRU (Not Recently Used) Page Replacement Algorithm

- For example if,
  - Page-0 is of class-2 (referenced, not modified)
  - Page-1 is of class-1 (not referenced, modified)
  - Page-2 is of class-0 (not referenced, not modified)
  - Page-3 is of class-3 (referenced, modified)

- So lowest class **page-2 needs to be replaced by NRU**.

Belady's Anomaly (in FIFO Page Replacement Algorithm)

- Page Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Page Faults of 4 Frame > Page Faults of 3 Frame

Three Frames

Page Request	1	2	3	4	1	2	5	1	2	3	4	5
Frame – 1	1	1	1	4	4	4	5	5	5	5	5	5
Frame – 2		2	2	2	1	1	1	1	1	3	3	3
Frame – 3			3	3	3	2	2	2	2	2	4	4
Page Faults (9)	F	F	F	F	F	F	F			F	F	

Four Frames

Page Request	1	2	3	4	1	2	5	1	2	3	4	5
Frame – 1	1	1	1	1	1	1	5	5	5	5	4	4
Frame – 2		2	2	2	2	2	2	1	1	1	1	5
Frame – 3			3	3	3	3	3	3	2	2	2	2
Frame – 4				4	4	4	4	4	4	3	3	3
Page Faults (10)	F	F	F	F			F	F	F	F	F	F

Belady's Anomaly (in FIFO Page Replacement Algorithm)

- Belady's anomaly is the **phenomenon in which increasing the number of page frames results in an increase in the number of page faults for certain memory access patterns**.

- This phenomenon is commonly **experienced when using the First-In First-Out (FIFO) page replacement algorithm**.

- In FIFO, the page fault may or may not increase as the page frames increase, but in Optimal and stack-based algorithms like LRU, as the page frames increase the page fault decreases.

Sum

- A computer has four page frames. The time of loading, time of last access and the R and M bit for each page given below. Which page FIFO, LRU and NRU will replace.

Page	Loaded	Last used	R	M
0	126	280	1	0
1	230	265	0	1
2	140	270	0	0
3	110	285	1	1

- **FIFO**:- Page which is **arrived first needs to be removed first**, so **page-3** needs to replace.

- **LRU**:- When page fault occurs, throw out the **page that has been used for the longest time**.
- **Page-1** is not used for the long time from all four, so LRU suggest replacing page-1.

- **NRU**:-
  - Page-0 is of class-2 (referenced, not modified)
  - Page-1 is of class-1 (not referenced, modified)
  - Page-2 is of class-0 (not referenced, not modified)
  - Page-3 is of class-3 (referenced, modified)
  - So lowest class **page-2** needs to be replaced by NRU

Working Set Model

- It is a concept which is used for page replacement.
- It is a good mix of local and global page replacement.
- In this scheme, we change the memory share of each process dynamically, which reduces the overall page faults for the process.

- **Δt:** It is the time after which each process frame's share is fixed.
- **Working Set:** Contains all the frames that are referred to in the window.
- **Window:** It is the total number of references checked at time Δt.

Note: Number of Elements or pages in the working set = Number of Frames given for that process.

- **Example:** Page references of process (P1 and P2)
  - Process (P1): $\underbrace{1, 2, 2, 1, 1, 2, 1, 2}_{\text{Window } (\Delta\text{t: 10 min})}$, 2, 3, 1, 4, 2, 5, 1, 2, 3, 1, 3

    Working Set: {1, 2}

  - Process (P2): 3, 4, 1, 6, 5, $\underbrace{2, 8, 1, 3, 4, 3, 4, 4}_{\text{Window } (\Delta\text{t: 10 min})}$, 3, 4, 3

    Working Set: {1, 2, 3, 4, 8}

Practice Questions

1. Explain multiprogramming with fixed partition.
2. Explain link list method for dynamic memory management.
3. How free space can be managed by OS.
4. Explain Swapping and Fragmentation in detail.
5. What is Paging? Explain paging mechanism in MMU with example.
6. Explain TLB and Virtual Memory.
7. Discuss demand paging.
8. Define following Terms: Thrashing
9. List different Page Replacement Algorithms? Discuss it in terms of page faults.
10. What is Belady's anomaly? Explain with suitable example.
11. Consider (7012030423032120170 1) page reference string: How many page fault would occur for following page replacement algorithm. Consider 3 frames and 4 frames.
    1. FIFO 2. LRU 3. Optimal

Practice Questions

12. Given six Partition of 300KB, 600KB, 350KB, 200KB, 750KB and 125KB(in order), how would the first-fit, best-fit and worst-fit algorithms places processes of size 115 KB, 500KB, 358KB, 200KB and 375KB(in order)? Which algorithm is efficient for the use of memory?
13. Calculate the page fault rates for below reference string in case of FIFO and Optimal page replacement algorithm. Assume the memory size is 4 page frames and all frames are initially empty. 0,2,1,6,4,0,1,0,3,1,2,1
14. Consider the following reference string. Calculate the page fault rates for FIFO and OPTIMAL page replacement algorithm. Assume the memory size is 4 page frame.

    1,2,3,4,5,3,4,1,6,7,8,7,8,9,7,8,9,5,4,5,4,2
15. Consider the page reference string: 1,2,3,4,5,3,4,1,6,7,8,7,8,9,7,8,9,5,4,5,4,2 With four Frames How many page faults would occur for the FIFO, Optimal page replacement algorithms? which algorithm is efficient? (assume all frames are initially empty)

Thank You